

Load Balancing, Capacity & Expert Collapse.

Left to itself a sparse router collapses: a few experts win every token, the rest starve, and capacity is wasted. We derive the Switch-Transformer auxiliary loss $\mathcal{L}_{\text{aux}} = \alpha E \sum_i f_i P_i$, prove it is minimised exactly at uniform load, define the per-expert *capacity* bound that drops overflow tokens, and work a balanced-vs-collapsed batch end to end. Every number on the following pages is computed in full — nothing summarised or deferred.

THE EQUATION

$$\mathcal{L}_{\text{aux}} = \alpha E \sum_{i=1}^E f_i P_i, \quad C = \text{capacity_factor} \cdot \frac{N}{E}$$

Worked example. $E=8$, a batch of $N=16$ tokens: compute f , P , $\sum f_i P_i$, $E \sum f_i P_i$, and $\mathcal{L}_{\text{aux}}$ at $\alpha=0.01$ for a balanced *and* a collapsed routing; then capacity C at factor 1.25 and a dropped-token scenario.

Topic 2 of 2 in this module · companion to the course page · v1.0

Contents

1	Why routing collapses	2
2	The auxiliary load-balancing loss	2
3	Worked example: balanced vs collapsed	3
4	Expert capacity and dropped tokens	4
5	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 Why routing collapses

Top- k routing (Topic 1) has no built-in pressure toward balance. Early in training the router slightly prefers a few experts; those experts get more tokens, hence more gradient, hence improve faster, hence get picked even more. This is a *rich-get-richer* feedback loop. At the limit a handful of experts handle nearly all tokens while the rest stay near their random initialisation — *expert collapse*. The collapsed experts are dead parameters, and the popular ones overflow their capacity and start dropping tokens. We need an explicit term that rewards spreading load evenly.

The router's gradient (Topic 1) only flows through experts that were *chosen*. A starved expert gets no gradient, so it can never “earn back” tokens on its own. Balance is not something the task loss will discover for free — it must be added as a separate objective that the optimiser actively trades against task quality.

2 The auxiliary load-balancing loss

Switch Transformer measures imbalance with two per-expert statistics over a batch of N tokens. Let Top_k be the selection of Topic 1 (Switch uses $k=1$). For expert i :

$$f_i = \frac{\#\{\text{tokens routed to } i\}}{N} \quad (\text{hard fraction, a count}), \quad P_i = \frac{1}{N} \sum_{t=1}^N p_{t,i} \quad (\text{mean router probability}).$$

Note $\sum_i f_i = k$ (each token contributes k picks) and $\sum_i P_i = 1$ (each row of p sums to 1). The auxiliary loss is their weighted dot product:

$$\mathcal{L}_{\text{aux}} = \alpha E \sum_{i=1}^E f_i P_i, \quad \mathcal{L} = \mathcal{L}_{\text{task}} + \mathcal{L}_{\text{aux}} \quad (1)$$

Symbol	Meaning	Range
f_i	fraction of batch tokens dispatched to expert i	$[0, 1], \sum f_i = k$
P_i	mean router probability assigned to expert i	$[0, 1], \sum P_i = 1$
α	loss weight (Switch uses 0.01)	scalar
E, N	#experts, #tokens in batch	scalar
C	per-expert capacity (max tokens)	scalar

2.1 Why the E factor, and why $f \cdot P$

Consider $k=1$, so $\sum_i f_i = 1$ and $\sum_i P_i = 1$. Both f and P are probability vectors over experts. The dot product $\sum_i f_i P_i$ is minimised, subject to those constraints, when both are uniform. Plugging $f_i = P_i = 1/E$:

$$\sum_{i=1}^E f_i P_i = \sum_{i=1}^E \frac{1}{E} \cdot \frac{1}{E} = E \cdot \frac{1}{E^2} = \frac{1}{E}, \quad \implies \quad E \sum_i f_i P_i = E \cdot \frac{1}{E} = 1.$$

So the E factor *normalises* the uniform optimum to exactly 1, making $\mathcal{L}_{\text{aux}} = \alpha$ at perfect balance regardless of how many experts there are. Any imbalance pushes the value above 1: because f and P are positively correlated (an expert chosen often also tends to have high mean probability), the product $f \cdot P$ penalises precisely that correlation — the mechanism that creates collapse. The hard count f measures true dispatch (but is non-differentiable), while the soft mean probability P carries the gradient into the router; multiplying them couples the two.

$\mathcal{L}_{\text{aux}} = \alpha E \sum_i f_i P_i$ bottoms out at the value α *iff* both the routed fraction f and the mean probability P are uniform ($= 1/E$). The non-differentiable count f supplies the “what actually happened” signal; the differentiable probability P supplies the gradient. Their product is the smallest possible only when load is perfectly spread.

3 Worked example: balanced vs collapsed

Take $E = 8$, $N = 16$ tokens, top-1 routing, $\alpha = 0.01$.

3.1 Balanced routing

Each expert is the top-1 choice of exactly $16/8 = 2$ tokens, and the router probabilities are near uniform. The per-expert statistics are

$$f_i = \frac{2}{16} = 0.125 \quad \forall i, \quad P_i \approx 0.125 \quad \forall i.$$

Then $\sum_i f_i P_i = 8 \cdot (0.125 \cdot 0.125) = 8 \cdot 0.015625 = 0.125$, so

$$E \sum_i f_i P_i = 8 \cdot 0.125 = 1.0, \quad \boxed{\mathcal{L}_{\text{aux}} = 0.01 \cdot 1.0 = \mathbf{0.0100}}$$

exactly the uniform optimum α .

3.2 Collapsed routing

Now suppose the router piles tokens onto the low-index experts. The top-1 counts are (5, 4, 3, 2, 1, 1, 0, 0) (summing to 16), and the mean probabilities track them. The two unused experts have $f_i = 0$.

expert i	1	2	3	4	5	6	7	8
count (top-1)	5	4	3	2	1	1	0	0
$f_i = \text{count}/16$	.3125	.2500	.1875	.1250	.0625	.0625	.0000	.0000
P_i (mean prob)	.2342	.1974	.1625	.1235	.0872	.0882	.0535	.0536
$f_i P_i$	.0732	.0494	.0305	.0154	.0054	.0055	.0000	.0000

Summing, $\sum_i f_i P_i = 0.17940$, so $E \sum_i f_i P_i = 8 \cdot 0.17940 = 1.43517$, and

$$\mathcal{L}_{\text{aux}} = 0.01 \cdot 1.43517 = \mathbf{0.01435}$$

The collapsed routing costs 43.5% more auxiliary loss than the balanced one (0.01435 vs. 0.0100), exactly as the term is designed to: the optimiser feels a gradient pushing it back toward uniform load.

3.3 The load heatmap

Plotting the per-expert token counts (cell intensity $\propto$ load, capped at indigo!80; the dashed line marks capacity $C = 3$):

	e1	e2	e3	e4	e5	e6	e7	e8
balanced	2	2	2	2	2	2	2	2
collapsed	5	4	3	2	1	1	0	0

The balanced row is a flat block; the collapsed row is a steep ramp with two dead experts (e7, e8) at the far end. The bold cells (e1, e2) exceed capacity — see the next section.

4 Expert capacity and dropped tokens

Hardware needs fixed-size buffers, so each expert accepts at most

$$C = \text{capacity_factor} \cdot \frac{N}{E} \quad (2)$$

tokens per batch. Tokens routed to an expert that is already full *overflow* and are *dropped*: they skip the MoE layer entirely, their residual connection $x \mapsto x$ carrying them through unchanged. For our batch, $N = 16$, $E = 8$, $\text{capacity_factor} = 1.25$:

$$C = 1.25 \cdot \frac{16}{8} = 1.25 \cdot 2 = 2.5 \longrightarrow \lceil 2.5 \rceil = \mathbf{3} \text{ tokens/expert.}$$

The factor 1.25 gives 25% headroom above the uniform mean $N/E = 2$. Applying $C = 3$ to the *collapsed* counts (5, 4, 3, 2, 1, 1, 0, 0):

expert i	1	2	3	4	5	6	7	8	total
routed	5	4	3	2	1	1	0	0	16
capacity C	3	3	3	3	3	3	3	3	—
dropped = $\max(0, \text{routed} - C)$	2	1	0	0	0	0	0	0	3

Experts 1 and 2 overflow, so **3** of the 16 tokens (18.75%) are dropped and pass through untransformed. Under the *balanced* routing every expert holds $2 \leq 3$, so *nothing* is dropped. Imbalance does not just raise $\mathcal{L}_{\text{aux}}$ — it actively destroys information by dropping tokens.

Capacity is a *per-batch* budget, set so the dispatch tensors have a fixed shape. The auxiliary loss and the capacity bound attack collapse from two sides: $\mathcal{L}_{\text{aux}}$ *discourages* imbalance via gradients during training, while C *contains* its damage by hard-capping any single expert's load. A well-trained router rarely overflows; capacity is the safety net, not the primary defence.

The capacity factor is a quality/compute knob with two failure modes. **Too low** (e.g. 1.0): $C = N/E$ leaves zero headroom, so even mild, harmless imbalance forces drops — here $C = 2$ would drop $(3, 2, 1, 0, \dots) = \mathbf{6}$ tokens, 37.5%, badly hurting quality. **Too high** (e.g. 2.0): buffers are padded to $C = 4$ slots/expert that mostly sit empty under balanced routing, wasting compute and memory. Typical values are 1.0–1.25 at train time and a little higher at inference.

5 Where this sits in the track

Back Module 7.1 T1 — routing, top- k gating, and the non-differentiable selection.
This Module 7.1 T2 — load-balancing loss, capacity, dropped tokens, expert collapse.
Next Phase 7 continues — scaling MoE: expert/data parallelism and routing at 1000+ experts.

A Reproducible (NumPy)

Inputs: $E=8$, $N=16$, $\alpha=0.01$; balanced top-1 counts (2×8); collapsed counts $(5, 4, 3, 2, 1, 1, 0, 0)$ with the mean probs in the table. Verified outputs: balanced $\mathcal{L}_{\text{aux}} = 0.0100$; collapsed $\sum f_i P_i = 0.17940$, $E \sum = 1.43517$, $\mathcal{L}_{\text{aux}} = 0.01435$; capacity $C = 1.25 \cdot 16/8 = 2.5 \rightarrow 3$; dropped = 3 tokens (collapsed, $C=3$); 6 tokens if $C=2$.

```
import numpy as np
E, N, alpha = 8, 16, 0.01

def L_aux(counts, P):
    f = np.asarray(counts) / N
    return (f*P).sum(), E*(f*P).sum(), alpha*E*(f*P).sum()

# balanced: 2 tokens each, near-uniform mean probs
cB = np.full(E, 2); PB = np.full(E, 0.125)
print(L_aux(cB, PB))      # (0.125, 1.0, 0.0100)
```

```
# collapsed
cI = np.array([5,4,3,2,1,1,0,0])
PI = np.array([.2342,.1974,.1625,.1235,.0872,.0882,.0535,.0536])
print(L_aux(cI, PI))      # (0.17940, 1.43517, 0.014352)

C = 1.25 * N / E          # 2.5 -> ceil 3
drop = np.maximum(cI - int(np.ceil(C)), 0)
print(C, drop, drop.sum()) # 2.5 [2 1 0 0 0 0 0 0] 3
```

Marevlo Research · Transformer Track · Module 7.1 · Topic 2 of 2.